

Making the EH Admin Console multi-tenant, on one shared database

Every client gets their own scanners, slides, users and feature toggles — like the enrichment-tool switches — while the app keeps running on a single database, one connection pool, one set of indexes.

Systems in scope eh-admin-service-main (Spring WebFlux) · eh-admin-ui-main (React/Redux)

Prepared 2026-08-07 **Status** Approved direction — pooled database

01 Executive summary

Today the admin console serves exactly one hospital client. There's no idea of "which client does this belong to" anywhere in the code — one MongoDB database, one LDAP login, one shared cache. To sell it to multiple clients, every piece of data needs to say who it belongs to, and every request needs to only see its own client's data.

The simplest way to do that is a `tenantId` field. Every record — every scanner, every slide, every config toggle — gets a `tenantId`. Every query the app makes automatically filters by it. One database keeps serving everyone, the same way it does today; the app is just taught to always ask "whose data is this?" before it reads or writes anything.

DECISION Keep one shared database for all clients. **Lowest cost, lowest complexity — one database, one connection pool, one set of indexes to manage**, instead of a separate database per client. A client's data is separated logically (by `tenantId`), not physically. If one specific client ever contractually requires their own database, the `tenantId` field makes that a data-export job later, not a rewrite — see §10.

What stays the same

- Same database (MongoDB), same collections, same Kubernetes deployment
- Same React/Redux frontend and component library
- Same Kafka + config-refresh mechanism
- Same LDAP login for hospitals that already use it

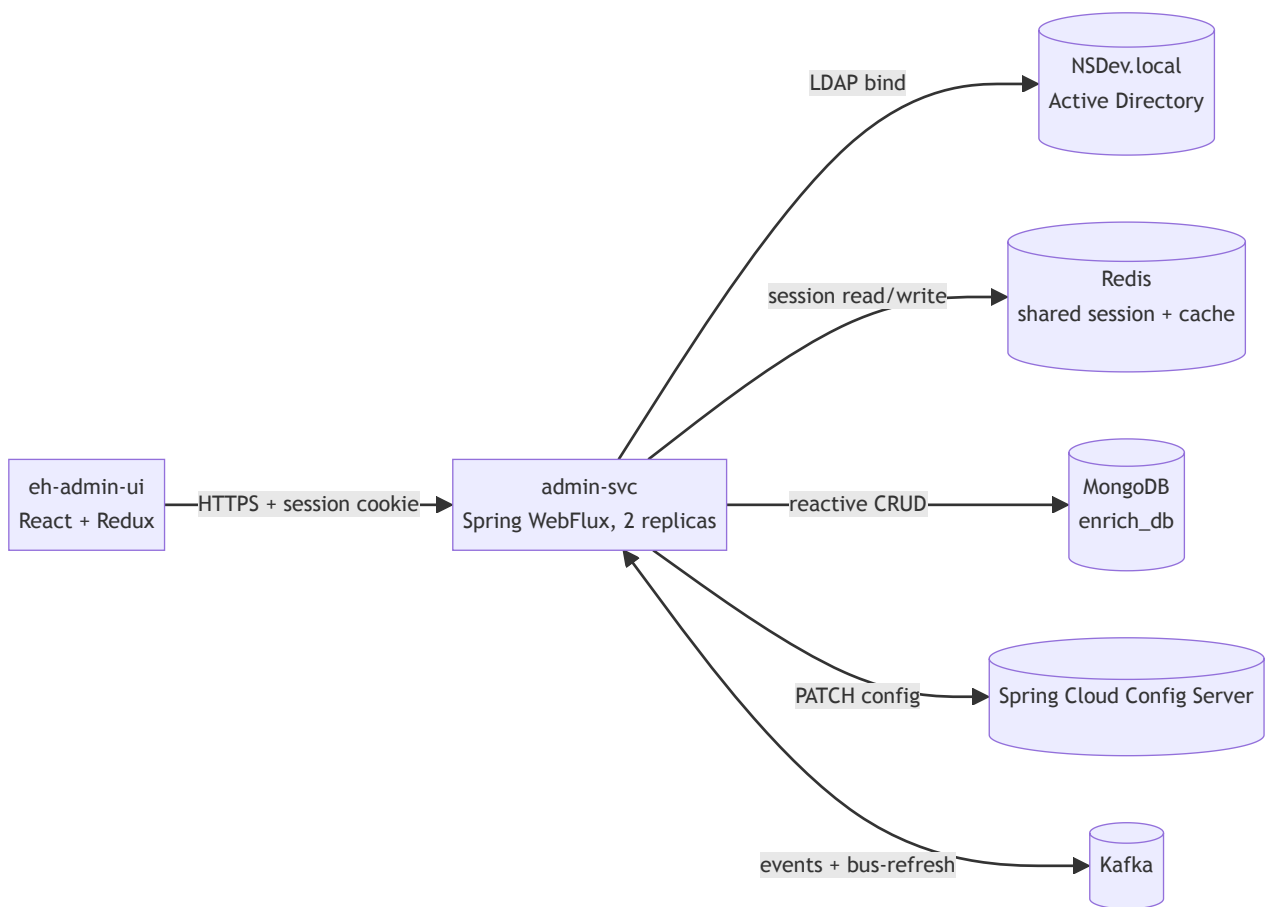
What's new

- A `tenantId` field on every record
- Every query/write automatically scoped to "the caller's own client"
- A simple `tenants` lookup table (id, name, status)
- Feature toggles (like enrichment-tool switches) become per-client instead of global

02 Current-state architecture

One `admin-svc` deployment (2 replicas) serving one client. Everything below is shared by every logged-in user, with nothing to tell them apart.

FIG. 1 – CURRENT, SINGLE-TENANT

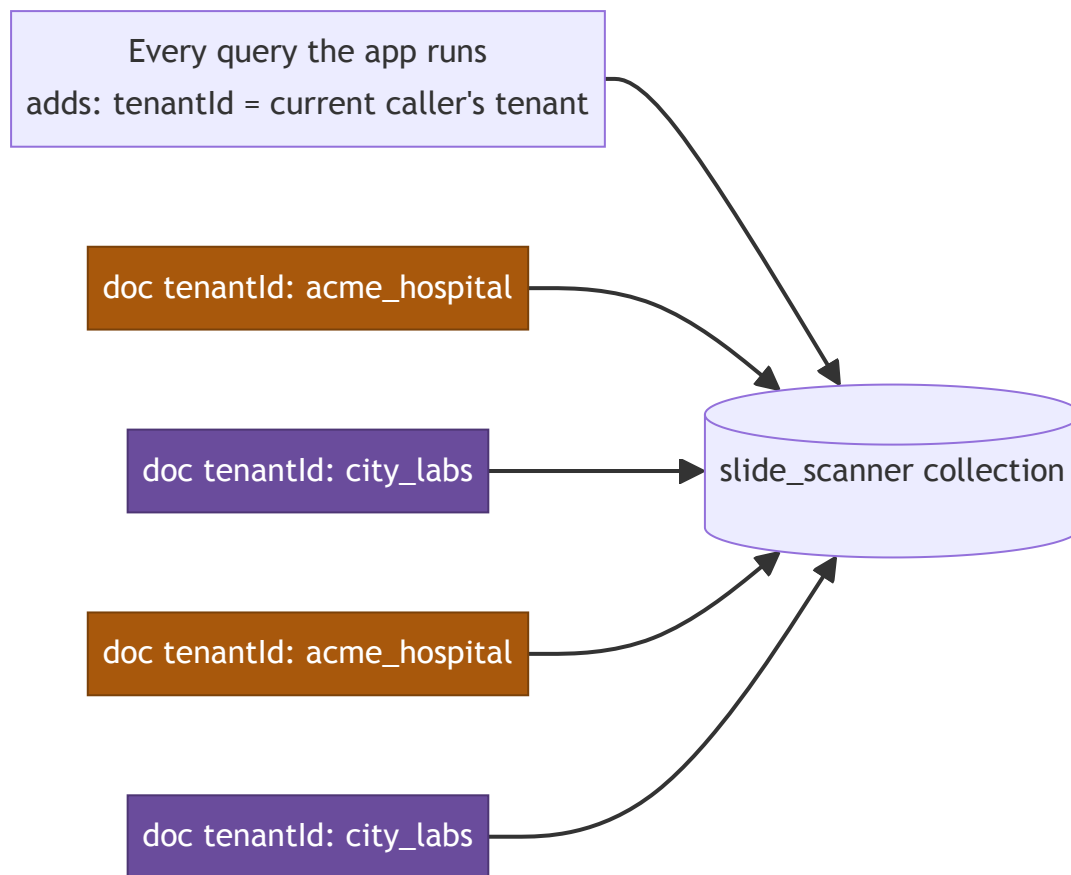


No record anywhere says which client it belongs to — `SlideScanner.hospitalName` is a free-text label, not a boundary.

03 One shared database, separated by `tenantId`

The core idea, in one picture: the same collection holds every client's documents, and every query adds "only mine."

FIG. 2 — ONE COLLECTION, MANY CLIENTS



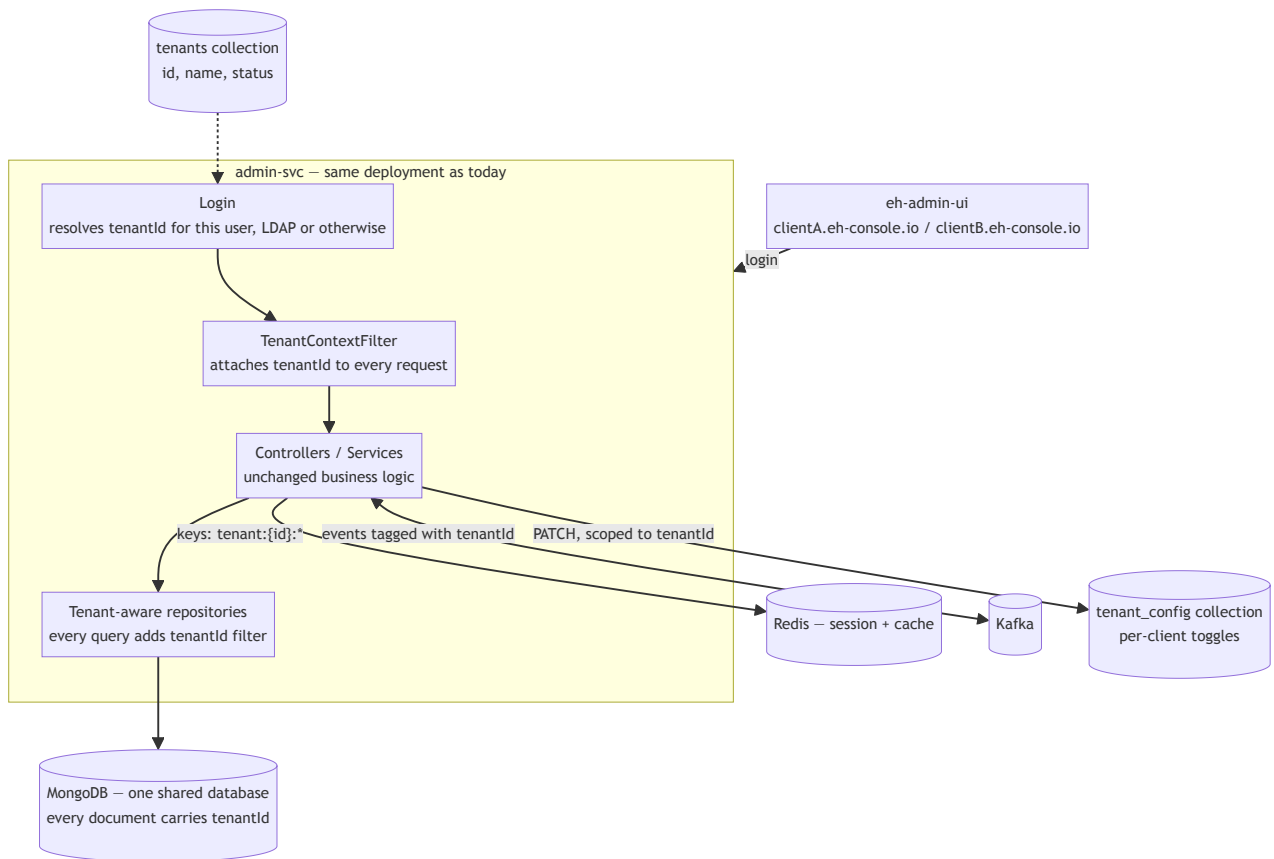
Acme Hospital and City Labs' scanners sit in the same collection. Acme's login can never see City Labs' orange-vs-purple records above, because the filter is applied before results ever leave the database.

This is the whole strategy. No routing to different databases, no per-client infrastructure to provision — just a field that's always present and always checked.

04 Target architecture

Two new pieces sit around the existing app: something that figures out *who's asking* (login), and something that makes sure every read/write is scoped to them (the tenant filter). The application code in between — controllers, services — barely changes.

FIG. 3 – TARGET TOPOLOGY

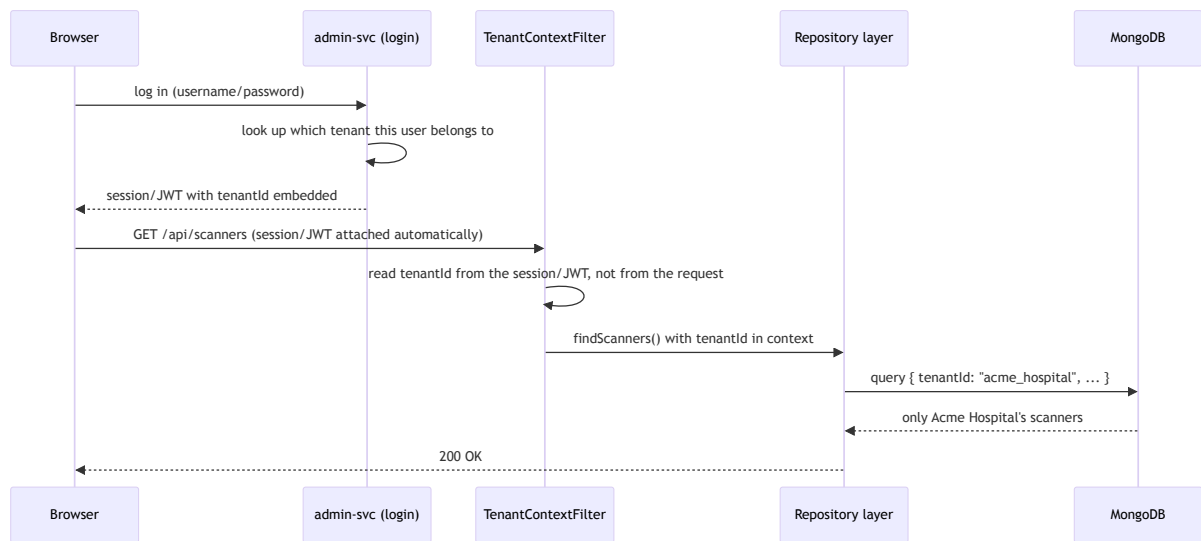


The only genuinely new infrastructure is the `tenants` lookup table and the `tenant_config` collection — both live in the same MongoDB database as everything else.

05 How a request flows

The mechanism that actually keeps clients apart: `tenantId` is decided once, at login, from something the browser can't fake — never taken from a URL parameter or request body.

FIG. 4 – A SINGLE REQUEST, END TO END



Because `tenantId` comes from the authenticated session, not from anything the client sends explicitly, there's nothing for a user to tamper with to see another client's data.

06 Data model changes

Every persisted entity gets a `tenantId` and an index that leads with it, so lookups stay fast as more clients are added.

ENTITY	COLLECTION	CHANGE
SlideScanner	slide_scanner	Add <code>tenantId</code> ; index becomes <code>(tenantId, deviceSerialNumber)</code> instead of a global unique field.
HospitalMetadata	hospital_metadata	Add <code>tenantId</code> — today this is a single global lookup despite the name; becomes genuinely per-client.
QaSlide	qa_slide	Add <code>tenantId</code> ; index <code>(tenantId, barcode)</code> .
SlideScanStatus	slide_scan_status	Add <code>tenantId</code> ; index <code>(tenantId, scanStatus)</code> and <code>(tenantId, slideBarcode)</code> .
SlideAnalysisReport	slide_analysis_reports	Add <code>tenantId</code> ; index <code>(tenantId, deviceSerialNumber)</code> .
DicomInstance	(dicom repo)	Add <code>tenantId</code> so DICOM store operations are scoped per client too.

Existing production data belongs to the current client. It gets backfilled with one fixed id (`t_default`) in a single migration (§8) — that client keeps working exactly as before, just with a `tenantId` now attached.

07 Login & per-client toggles

Login

- LDAP stays for hospitals that already use it — the login step just also looks up which `tenantId` that user/LDAP group belongs to.
- Clients without their own directory can be added later as plain username/password users tied to a `tenantId` , without changing the shared-database model.
- `tenantId` is attached to the session/JWT at login and never trusted from anywhere else (§5).

Per-client feature toggles

This is the direct answer to "the button enable/disable and other configurations should work per client." The closest existing example is the enrichment-tool export switches (`synapseEnabled` / `ibexEnabled` / `visioPharmEnabled`), which today write straight through to one shared Config Server.

STEP	TODAY	MULTI-TENANT
UI toggle	<code>EnrichmentConfig.tsx</code> switch, one global form	Same component; loads the current client's saved values on open
API call	<code>PATCH</code> <code>/api/enrichment/tools/{application}</code>	Same URL — <code>tenantId</code> comes from the session, not the path
Where it's stored	Forwarded straight to Config Server, no database record	Saved in a new <code>tenant_config</code> collection, keyed by <code>tenantId</code> + application
Propagation	<code>busRefresh()</code> over Kafka, all replicas reload	Same bus-refresh, now scoped so it only affects that one client

Any future toggle — a banner, a limit, a branding switch — follows this same pattern: write it to `tenant_config` keyed by `tenantId`, not a new global config key.

08 Code changes

Concrete changes, in the order they'd actually be written. Package names match the existing codebase (`com.eh.digiatalpathology.admin`).

1 – A place to hold "who's asking" (new file)


```
package com.eh.digiatalpathalogy.admin.tenant;

import reactor.core.publisher.Mono;
import reactor.util.context.Context;

public final class TenantContext {

    public static final String TENANT_ID_KEY = "tenantId";

    private TenantContext() {}

    // Reactor's Context replaces ThreadLocal here – WebFlux hops threads,
    // so a ThreadLocal would silently lose the tenant id mid-request.
    public static Mono<String> currentTenantId() {
        return Mono.deferContextual(ctx -> Mono.just(ctx.get(TENANT_ID_KEY)));
    }

    public static Context of(String tenantId) {
        return Context.of(TENANT_ID_KEY, tenantId);
    }
}
```

2 – Attach it to every request (new file)

```
package com.eh.digiatalpathalogy.admin.tenant;

import org.springframework.core.annotation.Order;
import org.springframework.security.core.context.ReactiveSecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ServerWebExchange;
import org.springframework.web.server.WebFilter;
import org.springframework.web.server.WebFilterChain;
import reactor.core.publisher.Mono;

@Component
@Order(-100) // after Spring Security resolves the authenticated principal
public class TenantContextFilter implements WebFilter {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, WebFilterChain chain) {
        return ReactiveSecurityContextHolder.getContext()
            .map(ctx -> (TenantPrincipal) ctx.getAuthentication().getPrincipal())
            .flatMap(principal ->
                chain.filter(exchange)
                    .contextWrite(TenantContext.of(principal.getTenantId())));
    }
}
```

`TenantPrincipal` wraps the existing authenticated user with the `tenantId` resolved at login (step 4 below).

3 – Add `tenantId` to an entity

FILE –

SRC/MAIN/JAVA/.../ENTITY/SLIDESCANNER.JAVA (modified)

```
@Document(collection = DbCollections.SLIDE_SCANNER)
@CompoundIndex(name = "tenant_serial_idx",
                def = "{ 'tenantId': 1, 'deviceSerialNumber': 1 }", unique = true)
public class SlideScanner {

    @Id
    private String id;

    private String tenantId;    // NEW – which client this scanner belongs to

    private String deviceId;
    private String deviceSerialNumber;
    private String name;
    private String hospitalName;
    // ...remaining fields unchanged
}
```

Repeat for `HospitalMetadata`, `QaSlide`, `SlideScanStatus`, `SlideAnalysisReport`, `DicomInstance`
— same one-field, one-index change each.

4 – Resolve tenant at login

FILE –

SRC/MAIN/JAVA/.../SERVICES/AUTHSERVICE.JAVA (modified)

```
// inside login(), after LDAP authentication succeeds:
return tenantRepository.findByLdapGroup(ldapGroup)
    .map(tenant -> new TenantPrincipal(ldapUser, tenant.getId()))
    .switchIfEmpty(Mono.error(new AccessDeniedException("No tenant mapped for this acco
```

One new lookup against the new `tenants` collection — everything else in `AuthService` is unchanged.

5 – Make repositories tenant-aware

FILE –

SRC/MAIN/JAVA/.../REPOSITORY/TENANTAWAREREPOSITORY.JAVA (new base class)

```
public abstract class TenantAwareRepository<T, ID> {

    private final ReactiveMongoOperations mongoOps;
    private final Class<T> entityType;

    protected TenantAwareRepository(ReactiveMongoOperations mongoOps, Class<T> entityTy
        this.mongoOps = mongoOps;
        this.entityType = entityType;
    }

    public Flux<T> findAllForTenant() {
        return TenantContext.currentTenantId()
            .flatMapMany(tenantId ->
                mongoOps.find(Query.query(Criteria.where("tenantId").is(tenantId)), ent
            }

    public Mono<T> findByIdForTenant(ID id) {
        return TenantContext.currentTenantId()
            .flatMap(tenantId -> mongoOps.findOne(
                Query.query(Criteria.where("_id").is(id).and("tenantId").is(tenantId)),
                entityType));
    }
}
```

`SlideScannerRepository` and the other five repositories extend this instead of calling `ReactiveMongoRepository.findAll()` directly. Existing custom query methods gain the same `.and("tenantId").is(tenantId)` clause.

FILE –

SRC/MAIN/JAVA/.../SERVICES/SLIDESCANNERSERVICE.JAVA (one-line change)

```
// before
return slideScannerRepository.findAll();

// after
return slideScannerRepository.findAllForTenant();
```

6 – Scope config toggles per client

FILE – SRC/MAIN/JAVA/.../CONTROLLER/ENRICHMENTTOOLCONTROLLER.JAVA (modified)

```
@PatchMapping("/{application}")
public Mono<Void> updateConfigurationByPath(@PathVariable String application,
                                           @RequestBody Map<String, Object> payload) {
    return TenantContext.currentTenantId()
        .flatMap(tenantId -> enrichmentToolService.updateAppConfig(tenantId, applicatio
    }
}
```

FILE – SRC/MAIN/JAVA/.../SERVICES/ENRICHMENTTOOLSERVICE.JAVA (modified)

```
public Mono<Void> updateAppConfig(String tenantId, String application, Map<String, Object> payload) {
    return tenantConfigRepository.upsert(tenantId, application, payload)
        .then(configurationClient.busRefresh(tenantId, application));
}
}
```

New `tenant_config` document shape: `{ tenantId, application, fields: { synapseEnabled, ibexEnabled, visioPharmEnabled }, updatedAt, updatedBy }`.

7 – Backfill existing data

FILE – SCRIPTS/BACKFILL-TENANT-ID.JS (new, run once)

```
const DEFAULT_TENANT = "t_default";
const collections = [
    "slide_scanner", "qa_slide", "hospital_metadata",
    "slide_scan_status", "slide_analysis_reports"
];

collections.forEach((name) => {
    db[name].updateMany(
        { tenantId: { $exists: false } },
        { $set: { tenantId: DEFAULT_TENANT } }
    );
});

db.slide_scanner.createIndex({ tenantId: 1, deviceSerialNumber: 1 }, { unique: true });
db.qa_slide.createIndex({ tenantId: 1, barcode: 1 });
db.slide_scan_status.createIndex({ tenantId: 1, scanStatus: 1 });
```

8 – Frontend: keep one client's data out of another's browser state

```
const storageKey = (tenantSlug: string, key: string) => `${tenantSlug}:${key}`;

export const persistSession = (tenantSlug: string, session: AuthSession) => {
  localStorage.setItem(storageKey(tenantSlug, "auth_user"), JSON.stringify(session.user));
  localStorage.setItem(storageKey(tenantSlug, "auth_scopes"), JSON.stringify(session.scopes));
};
```

Same Redux slices as today — persisted keys are just prefixed by tenant so switching accounts in one browser can't leave stale data behind. The API client (`apiClient.ts`) needs no changes: `tenantId` already rides the session cookie set at login.

09 Implementation plan

Five steps, each shippable on its own, none of which interrupt the current client.

1 Add `tenantId` everywhere — invisible change

Add the field + index to all six entities (§6), run the backfill script (§8.7) against production. Everything still behaves exactly as today — there's still only one tenant, `t_default`.

2 Wire tenant identity through login and repositories

Add `TenantContext`, `TenantContextFilter`, and `TenantPrincipal` (§8.1–2). Switch repositories to the tenant-aware base class (§8.5). Still one tenant behind the scenes — this just proves the plumbing works end to end.

3 Add the `tenants` lookup table

A small collection: id, name, status, LDAP group mapping (or login type). This is the entire "control plane" a pooled-database model needs — no infrastructure provisioning required.

4 Move config toggles to `tenant_config`

Point `EnrichmentToolController` — and any new enable/disable toggle — through the new tenant-scoped config path (§8.6) instead of the global Config Server write.

5 Onboard the second client

Insert a row in `tenants`, map their login (LDAP group or new user) to that `tenantId`. They start using the same app, same database, their own data — nothing to deploy or provision.

Later, once real usage exists: per-client rate limits and an audit log (who changed what) are worth adding, but neither blocks onboarding the first paying client.

10 Risks & tradeoffs of one shared database

- **Noisy neighbor:** a heavy query from one client can slow the database for everyone. Mitigate with query timeouts and keeping the `tenantId` -first indexes in place as data grows.
- **All-or-nothing backups:** a restore brings back every client's data at once. Acceptable at this stage; revisit if a client's contract requires independent recovery.
- **A missed filter is a real leak:** the tenant-aware repository base class (§8.5) exists specifically so no one has to remember to add the filter by hand — new repository methods should always go through it, not a raw `ReactiveMongoRepository` call.
- **If one client later needs their own database:** because every document already carries `tenantId`, moving them out is a filtered export/import, not a schema rewrite — the field pays for that flexibility even though nothing is siloed today.

11 Appendix – file reference map

eh-admin-service-main/src/main/java/.../tenant/TenantContext.java

New – holds the current request's tenant id (§8.1)

eh-admin-service-main/src/main/java/.../tenant/TenantContextFilter.java

New – attaches tenant id to every request (§8.2)

eh-admin-service-main/src/main/java/.../repository/TenantAwareRepository.java

New – base class every repository filters through (§8.5)

eh-admin-service-main/src/main/java/.../entity/*.java

Modified – `tenantId` field + compound index on all six entities (§6)

eh-admin-service-main/src/main/java/.../services/AuthService.java

Modified – resolves tenant id at login (§8.4)

eh-admin-service-main/src/main/java/.../controller/EnrichmentToolController.java

Modified – reference end-to-end toggle flow (§8.6)

eh-admin-service-main/scripts/backfill-tenant-id.js

New – one-time migration script (§8.7)

eh-admin-ui-main/src/store/slices/authSlice.ts

Modified – tenant-prefixed local storage keys (§8.8)

eh-admin-ui-main/src/components/features/enrichment/EnrichmentConfig/EnrichmentConfig.tsx
Unchanged UI – loads whichever client's config the session resolves to

EH Admin Console – Multi-Tenant Architecture & Implementation Plan · Draft
v2 · 2026-08-07